```
      leftTree ← constructKdTree(left, level +
1)                          #5
      rightTree ← constructKdTree(right, level +
1)              #6
    return new KdNode(median, leftTree,
rightTree, level)       #7
```

#1 Constructs a k-d tree from a set of nodes. We also pass the level, so we can recursively call this method for sub-trees.
#2 If `points` is empty, we need to create an empty node, so we just return `null`.
#3 If `points` has just one elements, we can create a leaf, a node without children, so recursion stops here.
#4 Otherwise, we first find the median of the set of points and its left and right partitions.
#5/#6 Recursively constructs k-d trees for the left and right partitions.
#7 Finally, the root of this tree is created, assigning the left and right subtrees previously created.

Listing 9.6 shows the algorithm to create a balanced k-d tree from a set of points. The principle is simple: the tree has to hold all the points in the set and, ideally, we would like left and right subtrees of the root to have the same number of elements. To achieve that, we can find the median in the set of points with respect to the first coordinate of the points, and use it as pivot for the root, having half of the remaining points that will end up on the root's left branch, and half on its right branch. But each of the branches of the root is a k-d tree itself, so it is possible to repeat the same step for root's